

# Informe del módulo de expansión y retroalimentación

CSDI RAG Engine

## 1 Módulo de expansión y retroalimentación

El módulo de expansión y retroalimentación es una capa complementaria sobre el recuperador híbrido del sistema. Su responsabilidad es enriquecer consultas y reutilizar señales de relevancia para mejorar la recuperación de información sin reemplazar a BM25, a la búsqueda vectorial ni a la combinación híbrida. El módulo opera después de que existen documentos fragmentados e indexados, y trabaja sobre los chunks recuperables del corpus técnico.

Su funcionamiento combina dos mecanismos. El primero es la expansión de consulta basada en pseudo-relevancia: cuando no existe feedback explícito suficiente, el sistema toma los primeros documentos de una recuperación inicial como documentos provisionalmente relevantes y extrae de ellos términos que pueden enriquecer la consulta original. El segundo mecanismo es el uso de feedback explícito almacenado: cuando existen valoraciones previas asociadas a una consulta o a consultas muy similares, el sistema puede ajustar el ranking mediante boosts o penalizaciones.

La salida del módulo no es una respuesta generada, sino una recuperación enriquecida y trazable. El sistema puede devolver consulta original, consulta expandida, términos añadidos, documentos usados como evidencia, feedback aplicado, puntuación original y puntuación ajustada. Esta trazabilidad permite analizar por qué un resultado aparece en una posición determinada y si la mejora proviene de pseudo-relevancia, de feedback explícito o de ambos.

El módulo se integró inicialmente en el Explorador de Búsqueda porque allí es posible comparar rankings, inspeccionar scores y capturar preferencias de manera controlada. La integración directa con el Chat queda como una extensión futura, ya que implicaría conectar el re-ranking por feedback al pipeline de generación RAG.

## 2 Fundamento técnico

La expansión de consultas busca compensar consultas breves o incompletas. En documentación técnica, una consulta como `decorators python` puede beneficiarse de términos presentes en documentos relevantes, como `wrapper`, `function`, `@decorator` o `higher order function`. En vez de usar diccionarios externos o listas genéricas de sinónimos, el módulo extrae términos desde el propio corpus, lo que mantiene la expansión alineada con el dominio real del sistema.

Cuando no hay feedback explícito, la estrategia usada es *pseudo-relevance feedback*. Esta técnica asume provisionalmente que los documentos mejor posicionados por una primera recuperación contienen vocabulario útil para reformular la consulta. Esa hipótesis no equivale a una validación humana: los documentos son pseudo-relevantes porque el sistema los colocó arriba en el ranking, no porque el usuario los haya marcado como útiles.

El feedback explícito es una señal distinta. En este caso, el usuario o el flujo A/B registra una valoración sobre la relación entre una consulta y un chunk. La escala utilizada es graduada:

| Valor | Interpretación |
|-------|----------------|
| 0     | No relevante   |
| 1     | Marginal       |
| 2     | Relevante      |
| 3     | Muy relevante  |

Esta escala permite distinguir documentos inútiles, parcialmente útiles y altamente relevantes. El feedback positivo puede promover resultados en búsquedas futuras, mientras que el feedback negativo puede reducir su peso. Por tanto, el módulo combina una señal automática provisional, la pseudo-relevancia, con una señal persistente y explícita, el feedback del usuario.

### 3 Flujo de funcionamiento

El flujo general depende de si existe feedback previo para la consulta. Si no existe, el módulo puede ejecutar una recuperación inicial, asumir como pseudo-relevantes los primeros resultados, extraer términos candidatos y construir una consulta expandida. Luego ejecuta la búsqueda usando esa consulta enriquecida.

Si sí existe feedback, el sistema puede consultar los registros asociados a la consulta normalizada o a consultas semánticamente similares. En ese caso, el módulo puede usar esa información para ajustar los resultados recuperados. Un chunk valorado como muy relevante puede recibir un boost; uno valorado como no relevante puede recibir una penalización. El ajuste complementa la puntuación original, no la sustituye por completo.

En el frontend, este flujo se utiliza dentro de una comparación A/B. El proceso es:

1. El usuario introduce una consulta en el Explorador de Búsqueda.
2. El frontend evalúa `query_feedback_comparison_probability` para decidir si muestra comparación.
3. Si no se activa, se ejecuta la búsqueda normal.
4. Si se activa, se consulta `/feedback?query=...` para saber si hay feedback previo.
5. Si no hay feedback, se compara búsqueda estándar contra búsqueda expandida.
6. Si hay feedback, se compara búsqueda estándar contra búsqueda con feedback.
7. El usuario elige entre *Prefiero A*, *Prefiero B*, *Ambas son útiles* o *Ninguna me sirve*.
8. La preferencia se persiste como feedback.

Esta separación mantiene intactos los modos existentes de búsqueda BM25, vectorial, híbrida y comparación tradicional. La comparación A/B solo se activa cuando corresponde según la configuración.

## 4 Persistencia y reutilización del feedback

El feedback se almacena en la tabla `query_feedback`. Cada registro relaciona una consulta con un chunk y una valoración de relevancia. Los campos principales son:

```
query
normalized_query
chunk_id
source_id
relevance
notes
session_id
created_at
updated_at
```

La consulta original se conserva en `query`, mientras que `normalized_query` permite reutilizar feedback aunque existan diferencias superficiales de mayúsculas, espacios o puntuación. Por ejemplo, `Python Decorators`, `python decorators` y `python decorators?` pueden resolverse a una forma normalizada común.

La reutilización puede ser exacta o semántica. La coincidencia exacta usa la consulta normalizada. La coincidencia semántica permite encontrar consultas históricas muy parecidas, pero con un umbral alto para evitar transferir feedback a preguntas distintas. Esto es importante porque un feedback válido para `python list comprehension` no necesariamente debe afectar una consulta sobre `python generator expression`.

El campo `notes` permite registrar el origen del feedback, por ejemplo si proviene de una comparación A/B. El campo `session_id` queda disponible para asociar señales a una sesión concreta en futuras extensiones. La persistencia en base de datos permite que el aprendizaje sobreviva reinicios y se acumule progresivamente.

## 5 Expansión y re-ranking

El endpoint `POST /api/v1/query-feedback/expand` permite ejecutar únicamente la expansión de una consulta. Devuelve `original_query`, `expanded_query`, `expansion_terms`, método utilizado y cantidad de documentos usados como evidencia. Aunque este endpoint es útil para trazabilidad y pruebas aisladas, el flujo principal no necesita llamarlo directamente porque la expansión también ocurre dentro de la búsqueda expandida.

La búsqueda expandida se ejecuta mediante `POST /api/v1/query-feedback/search`. Cuando `expansion_enabled` está activo, el servicio expande la consulta y luego recupera resultados. La respuesta incluye tanto los resultados como información de trazabilidad: consulta expandida, términos añadidos y documentos usados en la expansión.

El re-ranking por feedback se ejecuta mediante `POST /api/v1/query-feedback/search-with-feedback`. Este endpoint permite activar expansión, feedback exacto y feedback semántico. Su respuesta conserva la puntuación original y la puntuación ajustada:

```
original_score
```

```
adjusted_score
feedback_boost
feedback_applied
feedback_relevance
feedback_match_type
```

Con esta información se puede comprobar si un resultado fue promovido o penalizado por feedback. El diseño evita que el ajuste sea opaco: cada resultado puede indicar si se aplicó feedback, qué tipo de coincidencia se usó y qué relevancia histórica fue encontrada.

## 6 Comparación A/B e integración frontend

La comparación A/B es la interfaz principal para capturar preferencias. La opción A representa la búsqueda híbrida estándar. La opción B representa una estrategia mejorada: búsqueda con expansión cuando no existe feedback previo, o búsqueda con feedback cuando sí existe historial para la consulta.

| Caso         | Opción A         | Opción B              |
|--------------|------------------|-----------------------|
| Sin feedback | Híbrida estándar | Búsqueda expandida    |
| Con feedback | Híbrida estándar | Búsqueda con feedback |

El frontend se organiza en tres piezas. `queryFeedback.service.ts` encapsula las llamadas HTTP al backend. `useQueryFeedbackComparison.ts` concentra la lógica de activación, consulta de feedback previo, ejecución de estrategias y guardado de preferencias. `Search.tsx` integra el flujo en el modo de Exploración y renderiza dos columnas de resultados cuando la comparación se activa.

La configuración visible se agregó en `Settings.tsx`, dentro de *Retrieval y RAG*, bajo el bloque *Retroalimentación de búsqueda*. Allí se expone el parámetro `query_feedback_comparison_probabil` que controla la frecuencia de aparición de la comparación. Esta ubicación es coherente porque el módulo afecta el comportamiento de recuperación, junto a controles como Reranker, HyDE y cantidad de chunks.

## 7 API del módulo

El módulo expone su API bajo el prefijo `/api/v1/query-feedback`. Los endpoints principales son:

| Endpoint                   | Función                                                   |
|----------------------------|-----------------------------------------------------------|
| GET /health                | Verifica disponibilidad del módulo.                       |
| POST /expand               | Ejecuta solo expansión de consulta.                       |
| POST /search               | Ejecuta búsqueda con expansión opcional.                  |
| POST /search-with-feedback | Ejecuta búsqueda con expansión y re-ranking por feedback. |
| POST /feedback             | Guarda o actualiza feedback de una consulta y un chunk.   |
| GET /feedback?query=...    | Recupera feedback asociado a una consulta normalizada.    |
| GET /feedback/summary      | Devuelve estadísticas agregadas del feedback almacenado.  |

La separación de endpoints permite probar expansión, búsqueda, re-ranking y persistencia de forma independiente. El frontend usa `/feedback?query=...` para decidir qué comparación mostrar, `/search` para búsqueda expandida, `/search-with-feedback` para re-ranking y `/feedback` para persistir preferencias. El endpoint `/feedback/summary` está disponible para métricas agregadas, aunque no se muestra todavía como panel analítico.

## 8 Configuración operativa

El parámetro principal expuesto al usuario es:

```
query_feedback_comparison_probability
```

Sus valores van de 0.0 a 1.0. Un valor de 0.0 nunca muestra comparación, 0.25 la muestra ocasionalmente y 1.0 la muestra siempre, lo cual es útil para pruebas o demostraciones. Al guardarse desde Settings, el valor se normaliza a dos decimales antes de enviarse al backend.

El flujo de búsqueda con feedback también usa parámetros internos como `expansion_enabled`, `feedback_enabled`, `semantic_feedback_enabled`, `semantic_similarity_threshold`, `top_k_feedback` y `max_expansion_terms`. En la versión actual, algunos de estos valores se fijan desde el frontend para mantener un comportamiento conservador. Por ejemplo, la similitud semántica se usa con umbral alto para reutilizar feedback solo cuando las consultas son muy cercanas.

HyDE permanece como una configuración separada. Aunque también enriquece la búsqueda, su mecanismo es distinto: genera una hipótesis mediante LLM para mejorar la recuperación semántica. Query Feedback, en cambio, usa pseudo-relevancia y feedback histórico. Por eso ambos controles conviven en *Retrieval y RAG*, pero se describen de forma diferenciada.

## 9 Pruebas y validación

El backend incluye pruebas específicas para expansión, persistencia, re-ranking, servicio y rutas API:

```
test_query_feedback_expansion.py
test_query_feedback_repository.py
test_query_feedback_reranker.py
test_query_feedback_service.py
test_query_feedback_api_routes.py
```

Estas pruebas cubren expansión con documentos pseudo-relevantes, ausencia de evidencia suficiente, almacenamiento y actualización de feedback, consulta por query normalizada, resumen agregado, boosts positivos, penalizaciones negativas, coincidencia exacta, coincidencia semántica y validación de parámetros inválidos.

En frontend se validó compilación con `npm run build`, integración del hook en modo Exploración, preservación de Evaluación y Métricas, preservación de búsqueda normal y comparación tradicional, visualización de comparación A/B, guardado de preferencias y edición de la probabilidad desde Settings. También se verificó que `.env.local` y artefactos de `dist/` no fueran incluidos en commits.

## 10 Justificación y limitaciones

La solución cumple el objetivo del módulo porque introduce una capa de mejora sin sustituir los mecanismos base de recuperación. La expansión por pseudo-relevancia evita depender de diccionarios externos y aprovecha el propio corpus técnico. El feedback explícito aporta una señal humana persistente que puede ser más precisa que la inferencia automática del ranking.

La comparación A/B reduce la carga del usuario: en lugar de evaluar muchos documentos individualmente, compara dos variantes de ranking y guarda una preferencia simple. Esta preferencia se transforma en feedback reutilizable. Además, el diseño conserva trazabilidad mediante consulta expandida, términos añadidos, puntuación original, puntuación ajustada y tipo de coincidencia de feedback.

La principal limitación es que la primera integración está en el Explorador de Búsqueda y no en el Chat. Por tanto, el módulo mejora y valida flujos de recuperación, pero no modifica directamente la respuesta conversacional final. Una extensión futura consistiría en conectar `search-with-feedback` al pipeline `/api/v1/rag/query` para que el Chat use resultados reordenados por feedback antes de generar una respuesta.

Otra limitación es que la pseudo-relevancia depende de la calidad del ranking inicial. Si los primeros documentos no son realmente relevantes, los términos extraídos pueden no mejorar la consulta. Además, la comparación A/B solo registra feedback sobre resultados principales, no sobre todos los documentos mostrados. Finalmente, un feedback histórico incorrecto o sesgado puede afectar búsquedas futuras, por lo que la trazabilidad del ajuste resulta esencial.